



Politechnika Łódzka
Instytut Automatyki

Laboratorium Robotów Mobilnych

Instrukcja 6

Łódź, 2019

Wstęp

Celem niniejszych zajęć jest zaznajomienie się z autonomiczną nawigacją robota mobilnego.

Przygotowanie stanowiska

UWAGA!

Punkt nie dotyczy maszyny wirtualnej.

1. Należy połączyć się z dostępnym na stanowisku robotem mobilnym. Przebieg procedury nawiązywania połączenia i komunikacji z robotem opisano w instrukcji 5 w rozdziale "Uruchomienie rzeczywistego robota". Na komputerze powinien być uruchomiony roscore.
2. Na komputerze uruchamiamy niezbędne nody odpowiedzialne za manualne sterowanie robotem, tworzenie mapy, a następnie za jej zapisanie.

Uruchomienie sterowania robotem

```
roslaunch turtlebot3_teleop turtlebot3_teleop_key.launch
```

Uruchomienie mapowania - można wybrać inną metodę (gmapping, hector, karto) i rozdzielczość mapy. Można wykorzystać zdobytą wiedzę z laboratorium dotyczącą mapowania i działania poszczególnych metod.

```
roslaunch turtlebot3_slam turtlebot3_slam.launch slam_methods:=gmapping
```

Zapis mapy

```
roslaunch map_server map_saver -f ~/Ścieżka_do_naszego_folderu/nazwa_mapy
```

Autonomiczna nawigacja

UWAGA!

Weryfikacja działania możliwa w całości w symulacji. Wyniki i zachowanie robota można porównać z rzeczywistością.

Do testów algorytmów w symulacji należy uruchomić środowisko symulacyjne, ale przed tym lepiej dla pewności w oddzielnym terminalu uruchomić "roscore" a.

```
roslaunch roboty_mobilne simulation.launch
```

Uruchomienie launcha odpowiedzialnego za autonomiczną nawigację robota:

```
roslaunch turtlebot3_navigation turtlebot3_navigation.launch map_file:=$HOME/map.yaml
```

Należy wybrać odpowiednią mapę w zależności czy poruszamy się w środowisku symulacyjnym czy rzeczywistym.

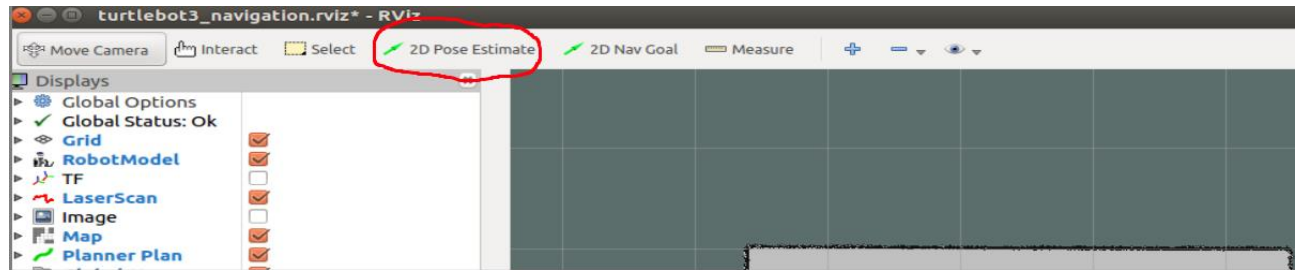
Aktualne położenie robota na mapie które należy wysłać do robota, aby autonomicznie dojechał do tego punktu można sprawdzić na kilka różnych sposobów. Oto niektóre z nich:

1. Sprawdzenie wartości w terminalu:

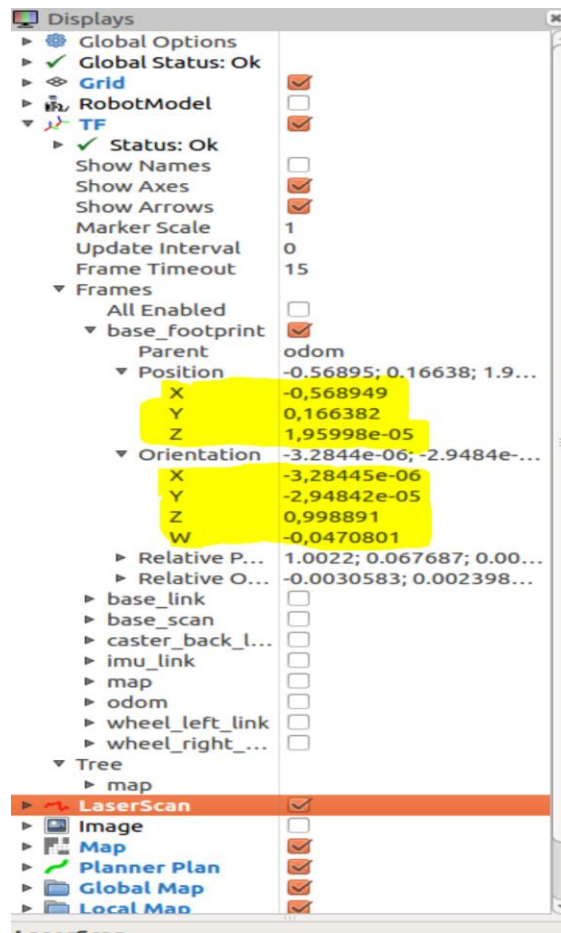
```
roslaunch tf tf_echo /odom /base_footprint
```

2. Opublikowanie punktu do którego ma dojechać robot z poziomu RVIZ'a i podgląd otrzymanych danych na topicu: /move_base_simple/goal

Najpierw należy podejrzeć topic w terminalu, a następnie wskazać punkt przy pomocy narzędzia *2D Pose Estimate*.



3. Odczyt aktualnego położenia robota z RVIZ'a. Dla tła `base_footprint` należy odczytać wartości z pól `Position` i `Orientation`



Wysyłanie celu do robota

Aby wysłać robota w dowolny punkt mapy należy opublikować położenie celu w układzie mapy na topicu `/move_base_simple/goal`

```

student@student-VirtualBox:~$ rostopic echo /move_base_simple/goal
WARNING: no messages received and simulated time is active.
Is /clock being published?
header:
  seq: 1
  stamp:
    secs: 1362
    nsecs: 582000000
  frame_id: "map"
pose:
  position:
    x: 0.560239136219
    y: -0.481483966112
    z: 0.0
  orientation:
    x: 0.0
    y: 0.0
    z: -0.642304961889
    w: 0.766449173744

```

Na topicu `/move_base/result` zwracana jest informacja o tym czy udało się osiągnąć zadany punkt czy też nie.

```

^Cstudent@student-VirtualBox:~$ rostopic echo /move_base/result
WARNING: no messages received and simulated time is active.
Is /clock being published?
header:
  seq: 2
  stamp:
    secs: 1480
    nsecs: 374000000
  frame_id: ''
status:
  goal_id:
    stamp:
      secs: 1476
      nsecs: 272000000
    id: "/move_base-3-1476.272000000"
  status: 3
  text: "Goal reached."
result:

```

Uwaga!

Jeśli w trakcie wykonywania zadań z jakiegoś powodu będą problemy z RVIZ'em (będzie się zaciął, niemożliwe będzie wysłanie komunikatu) do dalszych prac należy użyć dostarczonego skryptu `symulator_rezultatu`.

Zadanie 1

Napisać skrypt, który będzie publikował wiadomość (topic `move_base_simple/goal`) z jednym punktem do którego ma dojechać robot. Dla tego typu wiadomości w polu `frame_id` należy wpisać 'map', a w polu `seq` z każdym wywołaniem wysłania celu należy inkrementować o 1 wartość tego pola. Należy utworzyć odpowiedni licznik, który będzie miał domyślną wartość ustawioną na 1. Przed wykonaniem właściwej części programu można wykorzystać z biblioteki `time` funkcję `time.sleep(1)`, która wstrzymuje wykonanie dalszej części programu o 1s. Pozwoli to uniknąć sytuacji związanej z brakiem nawiązania połączenia z elementami ROS'a z poziomu jupyter notebook co może objawiać się niemożliwością po zrestartowaniu `kernela` wysłania po raz pierwszy celu.

Zadanie 2

Odebrać i wyświetlić rezultat dojazdu do punktu. Czy dojazd zakończył się pomyślnie czy też nie?. Wystarczy wyświetlenie tekstowego statusu, który jest zwracany w tej wiadomości. Wynik działania `move_base` publikowany jest na topicu `move_base/result`.

Zadanie 3

Rozszerzyć skrypt o możliwość ciągłego poruszania się pomiędzy 3-ma dowolnymi punktami. Również ma się wyświetlać informacja o tym czy zadanie udało wykonać się z powodzeniem lub nie. Jeśli nie udało się

dojechać do punktu to ponownie jest wysyłany ten sam cel. Jeśli po 3-ciej próbie wynik będzie negatywny to wysyłany jest kolejny punkt z listy. Komunikaty powinny informować m.in. o rezultacie dojazdu, do którego punktu aktualnie jedzie robot, jeśli ponawiane jest wysłanie punktu to również pojawia się informacja o tym która to próba z rzędu. W trakcie działania programu zakładamy, że nie zostanie wysłany żaden inny cel np. z poziomu RVIZ'a.

Przykładowe komunikaty jakie powinny się po sobie kolejno pojawiać:

```
Punkt 1 status: Goal reached
Punkt 2 status: Goal cancelled
Niepowodzenie dojazdu nr 1
Punkt 2 status: Goal cancelled
Niepowodzenie dojazdu nr 2
Punkt 2 status: Goal cancelled
Niepowodzenie dojazdu nr 3
3-krotne niepowodzenie dojazdu do punktu: 2
Jazda do następnego celu
Punkt 3 status: Goal reached
```

Zadanie dodatkowe

Zbadać wpływ różnych parametrów nawigacyjnych na zachowanie robota. W tym celu należy uruchomić `rqt_reconfigure`. Zmiana parametrów w oknie powoduje automatyczną zmianę parametrów i nie ma konieczności resetu oprogramowania.

Uruchomienie `rqt_reconfigure`

```
roslun rqt_reconfigure rqt_reconfigure
```